The SAS® 9 Local Services Management Utility (SAS_lsm)

Version 5

Installation, Configuration, and Usage Guide

DRAFT FOR TESTING - build 5.0.0-dev.3

Applies to SAS 9.4 deployments on UNIX and Linux

Contents

INTRODUCTION 3

UNDERSTANDING THE SAS_LSM SYSTEM ARCHITECTURE 3

Secure Shell (SSH) Considerations 4
Additional Considerations 4
SAS_lsm Tiers 5
SAS_lsm Tier Types 5
The SAS_lsm Configuration File 6

DEPLOYING SAS_LSM 9

Download and Install SAS_lsm 9
Set Up Passwordless SSH 10
Create the Configuration File 13
Review and Modify Your Configuration File 14
Validate the Configuration File 17
Check Every Host 17
Test a Dry Run 18

USEREXIT FRAMEWORK 18

Capabilities 18
Configuration 19
Using a SAS_lsm 4.x UserExit Directory 19
Custom Scripts 20

RUNNING SAS_LSM COMMANDS 21

Command Options 22
Exit Codes 22
How SAS_lsm Decides Whether a Tier Is Running 22

CENTRALIZED REPORT OUTPUT AND LOG COLLECTION 23

SCHEDULING SAS_LSM 24

SAS_lsm and cron 24
Starting SAS When the Machine Starts (Linux) 24

MIGRATING FROM SAS_LSM 4.X 25

What the convert Command Does 25
Changes in Behavior 26

TROUBLESHOOTING 27

APPENDIX: EXAMPLE CONFIGURATION FILES 28

Single Machine 28
Metadata, Compute, and Middle Tier 28
Metadata Cluster, Platform LSF Grid, and Middle Tier 29

APPENDIX: COMMAND REFERENCE 29

INTRODUCTION

Have you ever experienced any of the following issues?

- You try to restart SAS® servers to avoid loss of productivity, but the restart fails, causing delays.
- You struggle to find and analyze the right SAS logs when something goes wrong.

Or maybe you want other processes to help you, such as these:

- You want to run a process before or after you start or stop SAS services.
- You want SAS to notify you when something in the system stops working.

SAS Technical Support developed the SAS Local Services Management (SAS_lsm) utility to address these problems. SAS_lsm starts, stops, and checks the status of SAS services across every tier of a SAS 9.4 deployment on UNIX or Linux, in the correct order, from a single command on a single machine. It is available via SAS Note 58231, "Utility that manages single- or multi-tiered SAS® related services for UNIX and Linux deployments."

SAS_lsm 5 is a rewrite of the utility. It keeps the ordering and dependency checks that are the core of SAS_lsm, and changes how the utility is installed and configured:

- **One file to install.** SAS_lsm is now a single executable program. Nothing needs to be installed on the other machines in your deployment.
- **A simpler configuration file.** Deployments are described in a YAML file. Built-in *tier types* supply the commands and status checks for SAS servers, so most tiers need only a name and a list of host names.
- **Setup helpers.** New commands install SSH keys on every host with one password per host (**`ssh-setup`**, which also supports FIPS-enabled systems), draft a configuration file for you (**`discover`**), convert a SAS_lsm 4.x configuration file (**`convert`**), and check every host before the first run (**`doctor`**).
- **SSH in one direction only.** Passwordless SSH is needed only from the SAS_lsm controller to each host, not between all machines.
- **Several hosts per tier**, optionally controlled in parallel, plus built-in support for clustered SAS® Metadata Servers and Platform LSF grids.
- **Safer operation**: a dry-run mode that shows what would happen, protection against two runs at once, detailed exit codes, and a support bundle for SAS Technical Support when a tier fails.

Existing capabilities are retained: metadata horizontal clustering, control of SAS® LASR™ Analytic Server and third-party services, start-from and stop-at tier maintenance actions, centralized log collection and analysis, the UserExit framework, and e-mail notification.

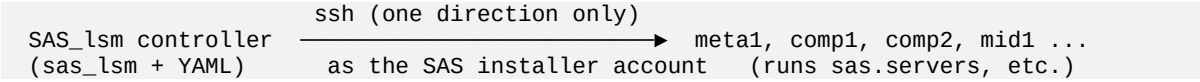
Note: If you are upgrading from SAS_lsm 4.x, read this guide through "Deploying SAS_lsm," and then see "Migrating from SAS_lsm 4.x." Your existing configuration file can be converted automatically.

UNDERSTANDING THE SAS_LSM SYSTEM ARCHITECTURE

SAS_lsm runs from a single machine (the *SAS_lsm controller*), and it runs commands on the other machines that host SAS services by using the secure shell (SSH) protocol. Passwordless SSH is required from the SAS_lsm controller to each machine, for the account that starts and stops SAS services on that machine.

You define machine names, tiers, and other settings in a SAS_lsm configuration file. A SAS_lsm controller can store any number of configuration files, so you can manage many SAS environments from one machine. SAS_lsm reads the configuration file that you specify and controls each tier in the proper order.

The SAS_lsm controller might be, but does not have to be, one of the machines in your SAS deployment. Many sites run SAS_lsm on their SAS Metadata Server host, because that machine is started first. When a tier runs on the controller itself under the same account that runs SAS_lsm, SAS_lsm runs the commands directly instead of through SSH. Other sites prefer a dedicated management host that is separate from SAS. Both methods work equally well.



Secure Shell (SSH) Considerations

SAS_lsm requires passwordless SSH from the SAS_lsm controller to every host where it runs commands. A single SAS_lsm action can run many commands on each host, so entering a password each time is not practical. SAS_lsm 5 includes a command, **`sas_lsm ssh-setup`**, that creates an SSH key and installs it on every host you list. You enter each account's password once. See "Set Up Passwordless SSH."

SSH is needed in **one direction only**: from the controller to each tier host. Tier hosts never connect to each other or back to the controller. This applies to Platform LSF grid tiers too, because the controller controls every grid node directly.

FIPS-enabled systems. If the SAS_lsm controller runs in FIPS mode, Ed25519 SSH keys cannot be used. The ssh-setup command detects FIPS mode automatically and uses an ECDSA or RSA key instead.

SSH behaves differently when commands are run in batch mode than in an interactive session. In batch mode, the system profile (for example, **/etc/profile**) is often not read. SAS_lsm uses batch mode only. If your SAS environment depends on variables set in a system profile, make sure that they are also set in the SAS installer account's own shell profile (for example, **${HOME}/.bashrc**). The **`sas_lsm doctor`** command helps you find this kind of problem.

SAS_lsm uses the operating system's own **`ssh`** command. Anything that works with **`ssh -o BatchMode=yes user@host`** also works with SAS_lsm, including host aliases, non-standard ports, and settings in **${HOME}/.ssh/config**. Sites that use Kerberos for SSH can use it instead of SSH keys.

The **`ssh_options`** setting in the configuration file lists options that SAS_lsm adds to every SSH command. For most sites, the default does not need to be changed. The default accepts the host key of a new host the first time SAS_lsm connects to it, and refuses to connect if that key changes later.

Additional Considerations

SAS recommends that you start and stop services with the SAS installer account that you used when you originally deployed SAS. Problems can occur if the account that starts SAS services does not have permission to the files. Files are owned by the SAS installer account, so using that account avoids such problems. SAS_lsm does not run as **root**.

Note: You can determine the SAS installer account by checking the ownership of a deployed SAS file such as *SAS-configuration-directory*/LevN/sas.servers.

SAS_lsm Tiers

A *tier* is a group of one or more machines whose services are started together. SAS_lsm starts tiers in the order in which they are listed in the configuration file and stops them in reverse order. A typical deployment has a metadata tier, a compute tier, and a middle tier, in that order.

Before it starts a tier, SAS_lsm checks that the tiers before it are running and the tiers after it are stopped. After it starts a tier, SAS_lsm waits until every host in the tier reports that its services are running before it starts the next tier.

SAS_lsm Tier Types

Each tier has a *type*. The type tells SAS_lsm which commands start, stop, and check the tier, and how to read the status output. SAS_lsm 5 has four tier types:

Tier Type	Use For	What You Must Provide
`sas-servers` (default)	Any machine whose SAS services are controlled by the sas.servers script: metadata (single server), compute, and middle-tier machines.	Host names and the SAS configuration directory
`metadata-cluster`	A clustered SAS Metadata Server with three or more nodes. Replaces the MDCN and MDCV tier types of SAS_lsm 4.x.	The host names of all cluster nodes and the SAS configuration directory
`lsf-grid`	The Platform LSF daemons of SAS® Grid Manager for Platform Suite.	The LSF master and node host names, and the LSF directories
`custom`	Anything else that has start, stop, and status commands: SAS LASR Analytic Server, third-party services, and so on.	The commands and the text that status output contains when the service is up or down

sas-servers. This is the tier type for most machines. SAS_lsm runs **sas.servers start**, **sas.servers stop**, and **sas.servers status** from the configuration directory that you set for the tier. A host is considered up when the status output contains **is UP** and no service reports **is NOT up**. A single (non-clustered) metadata server is a **sas-servers** tier.

metadata-cluster. SAS_lsm starts the metadata server on all nodes at the same time, by using **MetadataServer.sh**, and then waits until the cluster has a quorum. The cluster is considered up when a majority of nodes are running (or exactly half, including the first node listed). To stop the cluster, SAS_lsm runs **MetadataServer.sh stopCluster** on a running node, and then stops any node that is still running. You no longer need separate MDCN and MDCV tiers: one **metadata-cluster** tier replaces them. Only the metadata server process is controlled by this tier. If the metadata cluster nodes also run other SAS services, add a **sas-servers** tier for those machines later in the list.

lsf-grid. SAS_lsm starts and stops the Platform LSF daemons on the LSF master and each grid node, and optionally Process Manager. This replaces the **SAS_lsm_Grid_TEMPLATE.sh** UserExit script from SAS_lsm 4.x. See "Enable Control of a Platform LSF Grid."

custom. You supply the commands and the text that identifies a running or stopped service. See "Enable Control of Third-Party Services" and "Enable Control of SAS LASR Analytic Server."

The SAS_lsm Configuration File

The configuration file is a text file in YAML format, with four sections:

Section	Required?	Contents
version	Yes	Always 1.
settings	No	Settings for SAS_lsm itself, such as where reports are written and who receives e-mail.
defaults	No	Values that every tier uses unless the tier sets its own value, such as the SAS installer account and the SAS configuration directory.
tiers	Yes	The tiers of the deployment, in start order.

Here is a complete configuration file for a typical deployment with a metadata server, two compute machines, and a middle-tier machine:

```
version: 1

settings:
  status_root: /var/tmp/sas_lsm
  admin_email: [sasadmin@example.com]

defaults:
  user: sasinst
  configdir: /opt/sas/config/Lev1

tiers:
  - name: metadata
    hosts: [sasmeta01.example.com]

  - name: compute
    hosts: [sascomp01.example.com, sascomp02.example.com]
    parallel: true

  - name: midtier
    hosts: [sasmid01.example.com]
    configdir: /opt/sas/configmid/Lev1
    retries: 150
```

Each tier begins with a dash (**- name:**). The indentation is significant: settings that belong to a tier are indented under it. Use spaces, not tab characters.

The name of the configuration file matters: SAS_lsm uses it (without the .yaml extension) to name the directory where reports for that deployment are written. For example, reports for **prod.yaml** are written to *status_root*/prod/.

Settings

The **settings** section controls SAS_lsm itself. Every setting is optional.

Setting	Definition	Example Value
`status_root`	Root directory for reports, collected logs, and the lock file on the SAS_lsm controller. *Default: /var/tmp/sas_lsm*. Do not use /tmp, which might be cleared when the machine restarts.	`/usr/local/etc/SAS/reports`
`admin_email`	E-mail address or addresses of your deployment administrators. SAS_lsm sends the report after every start and stop action, and after a status action that finds a problem. Requires the mailx command on the controller.	`[sasadmin@example.com, oncall@example.com]`
`mail_command`	Command used to send e-mail. *Default: mailx*.	`mailx`
`archive_reports`	Keep a date-stamped copy of every start and stop report under *status_root/config-name*/archive/. *Default: true*.	`true`
`collect_logs`	When a tier fails, copy its recent log files to the controller, analyze them, and create a support bundle. Same as adding --collect-logs to every command. *Default: false*.	`true`
`log_minutes`	Collect only log files that changed within this many minutes. *Default: 60*.	`60`
`log_max_lines`	Number of lines to copy from the end of each log file. *Default: 5000*.	`5000`
`ssh_options`	Options added to every SSH command. *The default is suitable for most sites*.	`[-o, BatchMode=yes, -o, ConnectTimeout=10, -o, StrictHostKeyChecking=accept-new]`
`ssh_multiplex`	Reuse one SSH connection per host for each run, which makes runs much faster. Set to false if the home directory is on a file system that does not support this (some NFS file systems). *Default: true*.	`true`

Defaults and Tier Settings

These settings can be placed in the **defaults** section, where they apply to every tier, or in an individual tier, where they apply to that tier only.

> **Important:** Unlike SAS_lsm 4.x, a value that you set in one tier is **never** passed on to the next tier. Each tier uses its own value if it has one, and otherwise the value from the defaults section.

Setting	Definition	Example Value
`user`	Account that runs the commands for the tier (the SAS installer account). *Default: the account that runs SAS_lsm*.	`sasinst`

Setting	Definition	Example Value
configdir	SAS configuration directory, including the Lev*N* directory. Required for sas-servers and metadata-cluster tiers.	**/opt/sas/config/Lev1**
parallel	Control all hosts of the tier at the same time instead of one at a time. *Default: false.*	**true**
retries	How many times SAS_lsm checks the status of a tier while it waits for the tier to start or stop. *Default: 10.*	**30**
retry_interval	Seconds between status checks. *Default: 6.*	**10**
tier_delay	Seconds to wait after a tier has started before starting the next tier. *Default: 15.*	**15**
command_timeout	Seconds before SAS_lsm stops waiting for a single command to finish. *Default: 900 (15 minutes).*	**1800**
ok_exit_codes	Exit codes of start, stop, and status commands that SAS_lsm treats as successful. The tier's state is judged from the status output, so this rarely needs to change. *Default: [0, 1, 2, 4].*	**[0, 1, 2, 4]**

SAS_lsm waits up to approximately *retries* × *retry_interval* seconds for a tier to start or stop. The defaults allow one minute, which is enough for most metadata and compute tiers. Middle-tier web application servers can take 10 to 20 minutes to start, so increase **retries** for the middle tier (for example, **retries: 150** with the default interval allows 15 minutes).

Tier Definitions

These settings define a tier. Only **name** and **hosts** are needed for most tiers.

Setting	Description	Notes
name	A short name for the tier, such as metadata or midtier. Used in messages and in commands such as sas_lsm start --from midtier.	Required. Must be unique.
type	sas-servers, metadata-cluster, lsf-grid, or custom.	Optional. Default: sas-servers.
hosts	Host names of the machines in the tier, in square brackets and separated by commas.	Required, except for lsf-grid tiers.
start **stop** **status**	Commands that start, stop, and check the tier.	Required for custom tiers. Optional for other types (overrides the built-in command).
restart	Command that restarts the tier.	Optional. If not set, SAS_lsm stops and then starts the tier.

Setting	Description	Notes
`up_pattern` `down_pattern`	Text in the status output that shows a service is up or down.	Required for custom tiers.
`logdir`	Directory that contains the tier's log files, for log collection.	Optional. Default: the tier's configdir.
`userexits`	Scripts to run before or after the tier is started, stopped, or checked.	Optional. See "UserExit Framework."
`userexit_dir`	A SAS_lsm 4.x-style UserExit script directory on the tier's first host.	Optional.
`lsf`	Platform LSF settings.	Required for lsf-grid tiers.

In commands, you can use **{configdir}** and **{user}**, which are replaced by the tier's values, and **{host}**, which is replaced by the name of the host that the command runs on.

For host names, the fully qualified domain name (FQDN) is recommended. A short name or alias is acceptable if the SAS_lsm controller can resolve it.

DEPLOYING SAS_LSM

Perform every step in this section as the SAS installer account, on the machine that you have chosen as the SAS_lsm controller. The steps are:

1. Download and install SAS_lsm.
2. Set up passwordless SSH to your SAS machines.
3. Create a configuration file.
4. Review and validate the configuration file.
5. Check every host with the doctor command.
6. Test a dry run, and then run SAS_lsm.

Download and Install SAS_lsm

SAS_lsm 5 is distributed as one executable file for each operating system. Choose the file that matches the SAS_lsm controller (not the other machines in your deployment):

SAS_lsm Controller Operating System	File Name
Linux on x86-64 (Red Hat Enterprise Linux, SUSE, Oracle Linux, and others), including FIPS-enabled systems	`sas_lsm-linux-amd64`
Linux on ARM64	`sas_lsm-linux-arm64`
Linux on IBM POWER (little endian)	`sas_lsm-linux-ppc64le`
IBM AIX 7.2 or later	`sas_lsm-aix-ppc64`
Oracle Solaris 11 on x86-64	`sas_lsm-solaris-amd64`

Note: Solaris on SPARC and HP-UX are not supported as SAS_lsm controllers. If your SAS deployment runs on these platforms, you can run the SAS_lsm controller on a Linux machine that can reach your SAS machines by SSH, or continue to use SAS_lsm 4.x.

To install SAS_lsm:

1. Download the file for your platform from the SAS_lsm GitHub project page. Under the newest release, locate the **Assets** list.
2. Transfer the file to the SAS_lsm controller.
3. Create a directory for SAS_lsm, copy the file into it with the name sas_lsm, and make it executable:

```
mkdir -p /opt/sas/utilities/sas_lsm
cp sas_lsm-linux-amd64 /opt/sas/utilities/sas_lsm/sas_lsm
chmod 755 /opt/sas/utilities/sas_lsm/sas_lsm
```

4. Verify the installation:

```
/opt/sas/utilities/sas_lsm/sas_lsm version
```

Do not place SAS_lsm in the Utilities directory under your SAS configuration directory, because SAS deployment tools manage that directory.

Optionally, add the SAS_lsm directory to the SAS installer account's PATH, and set a default configuration file so that you do not need to type -c on every command:

```
echo 'export PATH=/opt/sas/utilities/sas_lsm:$PATH' >> ~/.bash_profile
echo 'export SAS_LSM_CONFIG=/opt/sas/utilities/sas_lsm/prod.yaml' >> ~/.bash_profile
```

The examples in the rest of this guide assume that sas_lsm is in your PATH.

Set Up Passwordless SSH

If every tier in your deployment runs on the SAS_lsm controller under the account that runs SAS_lsm (a single-machine deployment), you can skip this step.

Otherwise, run the ssh-setup command with the host names of every machine in your SAS deployment. You do not need a configuration file yet:

```
sas_lsm ssh-setup --user sasinst \
    --hosts sasmeta01.example.com,sascomp01.example.com,sasmid01.example.com
```

If a machine uses a different account, write it as *account@host*, for example **sasmid@sasmid01.example.com**. After you have a configuration file, you can also run **sas_lsm ssh-setup -c prod.yaml** to set up every account and host in that file.

The ssh-setup command performs these steps:

1. **Chooses an SSH key.** It uses an existing key of the account, or creates a new key without a passphrase. On FIPS-enabled systems, it never uses Ed25519 keys (see "SSH Keys and FIPS Mode").
2. **Checks each host** to see whether it already accepts the key, and lists each host with its state (see the table below).
3. **Explains what happens next** before it asks for anything: how many hosts need the key, and that you will be asked for each account's password once.

4. **Installs the key** on each host that needs it. You are prompted for that account's password; the prompt names the host and the account.
5. **Verifies** that each host now accepts the key, and prints a summary.

Each host is listed with one of these states:

State	Meaning	What Happens Next
[ready]	The key is already installed.	Nothing. The host is skipped.
[needs key]	The host can be reached, but the key is not installed yet. This is expected the first time that you run ssh-setup.	You are asked for the account's password once, and the key is installed.
[problem]	The host name cannot be resolved, or the host or its SSH server cannot be reached.	The host is skipped, and a suggested fix is shown. A password would not help.
[host key]	The host's SSH key has changed since it was first recorded.	The host is skipped. Confirm with your system administrator that the change is expected, run ssh-keygen -R *host*, and then run ssh-setup again.

Example output from a first run:

```
Using existing SSH key /home/sasinst/.ssh/id_ecdsa.pub

Checking which hosts already accept this key...

  [ready]      sasinst@sasmeta01.example.com   key already installed
  [needs key]  sasinst@sascomp01.example.com   key not installed yet (expected)
  [needs key]  sasinst@sasmid01.example.com    key not installed yet (expected)

2 hosts need the key. For each one, ssh-copy-id will ask for that account's
password once, then install the key so SAS_lsm can log in without a password.
Continue? [y/N] y

==> [1/2] sasinst@sascomp01.example.com: enter the password for sasinst when prompted
sasinst@sascomp01.example.com's password:
Number of key(s) added: 1
    verified: sasinst@sascomp01.example.com now accepts the key

==> [2/2] sasinst@sasmid01.example.com: enter the password for sasinst when prompted
sasinst@sasmid01.example.com's password:
Number of key(s) added: 1
    verified: sasinst@sasmid01.example.com now accepts the key

Summary
  ready:    3 host(s) (1 already set up, 2 installed now)

All hosts are ready.
```

It is safe to run ssh-setup again. Hosts that are already set up are skipped, so after you fix a host marked [problem] or [host key], simply run the same command again. The command ends with exit code 0 when every host is ready, and 1 if any host was skipped or failed.

The ssh-setup command has these options:

Option	Description
`--hosts H1,USER@H2,...`	The hosts to set up. Use this option or -c, not both.
`--user USER`	The account for hosts that are listed without USER@. Default: the account that runs SAS_lsm.
`-c FILE`	Set up every account and host in a configuration file.
`--key-type TYPE`	auto (default), ecdsa, rsa, or ed25519. See "SSH Keys and FIPS Mode."
`--yes`	Do not ask for confirmation before installing keys. You are still prompted for passwords.
`--verbose`	Also show the SSH program's own message for each host that is not ready. Useful when a host is marked [problem].

SSH Keys and FIPS Mode

The ssh-setup command checks whether the SAS_lsm controller runs in FIPS mode, by reading /proc/sys/crypto/fips_enabled and, on RHEL 8 and later, the system crypto policy. In FIPS mode, Ed25519 keys are not allowed, so the key is chosen as follows:

--key-type	Not in FIPS Mode	In FIPS Mode
`auto (default)`	Uses or creates an Ed25519 key, then ECDSA, then RSA	Uses or creates an ECDSA (P-384) key, then RSA (4096 bits). Ed25519 is never used.
`ecdsa or rsa`	That key type	That key type
`ed25519`	Ed25519	Refused with an error message

An existing private key is never overwritten. If the file ~/.ssh/id_ed25519 already exists on a FIPS-enabled system, it is left in place, and the ssh command might print the harmless warning "Ed25519 keys are not allowed in FIPS mode." To stop the warning, move that key file to another directory. SAS_lsm uses the ECDSA or RSA key automatically.

Setting Up SSH Manually

If the ssh-copy-id command is not available on the SAS_lsm controller (for example, on some AIX and Solaris systems), ssh-setup tells you so before it asks for any passwords. Install the key manually with these commands. On FIPS-enabled systems, use **-t ecdsa -b 384** instead of **-t ed25519**.

```
# On the SAS_lsm controller, as the SAS installer account (once):
ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub              # copy this line

# On each tier host, as the tier's account:
mkdir -p ~/.ssh && chmod 700 ~/.ssh
echo "paste-the-public-key-line-here" >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys

# Back on the controller - this must print the host name without a password prompt:
ssh -o BatchMode=yes sasinst@sascomp01.example.com hostname
```

Create the Configuration File

There are three ways to create your configuration file. Choose the one that fits your situation:

Method	Use When
A. Convert your SAS_lsm 4.x configuration file	You already use SAS_lsm 4.x.
B. Discover your deployment (recommended for new users)	You are new to SAS_lsm, and your SAS deployment uses standard SAS configuration directories.
C. Start from an example file	You prefer to write the file yourself, or your deployment is unusual.

Method A: Convert a SAS_lsm 4.x Configuration File

The convert command reads your SAS_lsm 4.x configuration file and writes an equivalent SAS_lsm 5 file. It also lists anything that you should review:

```
sas_lsm convert /opt/sas/utilities/SAS_lsm.cfg > /opt/sas/utilities/sas_lsm/prod.yaml
```

The convert command requires the BASH shell, because a SAS_lsm 4.x configuration file is a BASH script. It reads the file in the same way that SAS_lsm 4.x did. See "Migrating from SAS_lsm 4.x" for details of what is converted.

Method B: Discover Your Deployment

The discover command connects to each machine that you list, looks in the SAS configuration directory to see which SAS components are there, and writes a draft configuration file with the tiers in the usual order: metadata, compute, and then middle tier. It does not change anything on the machines. Passwordless SSH must already work, so run ssh-setup first (see "Set Up Passwordless SSH").

You need the following information before you run the command:

Value Required	Example Answer	Notes
The host name of each machine in the SAS deployment	**`sasmeta01.example.com, sascomp01.example.com, sasmid01.example.com`**	Separate the names with commas. Use the same list that you gave to ssh-setup.
The SAS configuration directory	**`/opt/sas/config/Lev1`**	Include Lev N. If the middle tier uses a different directory, fix that tier in the draft afterward.
The SAS installer account	**`sasinst`**	See "Additional Considerations."
The path and name of the configuration file	**`/opt/sas/utilities/sas_lsm/prod.yaml`**	The file name is used to name the report directory.

```
sas_lsm discover --hosts
sasmeta01.example.com,sascomp01.example.com,sasmid01.example.com \
                 --configdir /opt/sas/config/Lev1 --user sasinst \
                 > /opt/sas/utilities/sas_lsm/prod.yaml
```

Machines that could not be reached, or that have no SAS services in the configuration directory, are listed as comments at the end of the file. Review the draft carefully: discovery bases its decisions on which directories exist, and it cannot know about services that are not under the SAS configuration directory (for example, SAS LASR Analytic Server or third-party services).

Method C: Start from an Example File

Copy the example in "The SAS_lsm Configuration File" (earlier in this guide), or one of the examples in the "Example Configuration Files" appendix, and change the host names, account, and directory to match your deployment.

Review and Modify Your Configuration File

Open the configuration file in a text editor such as vi or nano, and check it for accuracy. Pay particular attention to these items:

- The tiers are in the order in which they must start.
- The host names are correct and can be resolved from the SAS_lsm controller.
- The **user** and **configdir** values are correct for every tier.

If you add hosts or accounts that you did not give to ssh-setup, run **sas_lsm ssh-setup -c prod.yaml**. Hosts that are already set up are skipped.

For most standard deployments, no further changes are needed. The following sections describe common changes.

Use a Different Configuration Directory or Account for One Tier

If one tier uses a different SAS configuration directory or account (for example, a middle tier installed in /opt/sas/configmid/Lev1), set the value in that tier only. Other tiers continue to use the value in the defaults section:

```
defaults:
  user: sasinst
  configdir: /opt/sas/config/Lev1

tiers:
  - name: metadata
    hosts: [sasmeta01.example.com]
  - name: compute
    hosts: [sascomp01.example.com]
  - name: midtier
    hosts: [sasmid01.example.com]
    configdir: /opt/sas/configmid/Lev1     # this tier only
```

Two Tiers on One Machine

If one machine hosts two tiers that must be controlled separately, such as a shared compute and middle-tier server with two configuration directories, list the same host name in both tiers and set a different **configdir** in each.

Control Several Machines at Once

A tier can contain several machines. By default, SAS_lsm controls them one at a time. Add **parallel: true** to control them at the same time, which is faster. Do not use parallel for machines that must start in a particular order, such as a clustered middle tier, unless you know that this is safe for your deployment.

Enable Control of a Clustered SAS Metadata Server

Use one tier of type metadata-cluster that lists every node, with the first-deployed node first:

```
  - name: metadata
    type: metadata-cluster
    hosts: [sasmeta01.example.com, sasmeta02.example.com, sasmeta03.example.com]
```

A configuration file can contain only one metadata-cluster tier, and it must list at least three hosts.

Enable Control of a Platform LSF Grid

Add a tier of type lsf-grid, and supply the LSF details in an **lsf** section. Do not list **hosts** for this tier type.

```
  - name: lsf
    type: lsf-grid
    parallel: true
    lsf:
      master: gridctl.example.com
      nodes: [node1.example.com, node2.example.com, node3.example.com]
      envdir: /opt/sas/lsf/conf
      serverdir: /opt/sas/lsf/10.1/linux2.6-glibc2.3-x86_64/etc
      pm_host: gridctl.example.com
      pm_envdir: /opt/sas/pm/conf
      pm_serverdir: /opt/sas/pm/10.2/linux2.6-glibc2.3-x86_64/etc
      ego_controls_daemons: false
      sudo: true
```

LSF Setting	Description
master	Host name of the LSF master. Required.
nodes	Host names of the other LSF hosts.
envdir	The LSF_ENVDIR directory, which contains profile.lsf. Required.
serverdir	The LSF_SERVERDIR directory, which contains the LSF daemon programs. Required.
pm_host, pm_envdir, pm_serverdir	Optional. The Process Manager host, its JS_ENVDIR directory (which contains profile.js), and its JS_SERVERDIR directory.
ego_controls_daemons	Set to true if EGO starts the res and sbatchd daemons. SAS_lsm then starts only the LIM daemon on each host. Default: false.
sudo	Run LSF commands through sudo. The LSF daemons run as root, so this is usually required. Default: true.

To start the grid, SAS_lsm starts LSF on the master, then on each node, and then starts Process Manager. To stop the grid, it reverses the order. SAS_lsm checks the status of the grid by looking for the LSF daemon processes on each host.

Because the LSF daemons run as root, the SAS installer account must be able to run the LSF commands with sudo without a password. Ask your system administrator to add a rule like the following one to the sudoers file on every LSF host (adjust the paths to match your LSF directories):

```
sasinst ALL=(root) NOPASSWD: /bin/sh -c . /opt/sas/lsf/conf/profile.lsf *, \
                             /bin/sh -c . /opt/sas/pm/conf/profile.js *
```

On some older Linux releases, you must also add **Defaults:sasinst !requiretty**. The doctor command checks that passwordless sudo works.

The lsf-grid tier controls only the LSF daemons. Put the SAS services on the grid machines (for example, the object spawner) in a separate sas-servers tier after the lsf-grid tier. Process Manager under EGO control (egosh service start ProcessManager) is not built in. If you need it, use a UserExit script or a custom tier.

Enable Control of Third-Party Services

Use a tier of type custom. You provide the commands and the text that the status command prints when the service is up and when it is down. The text is a regular expression, and it is case-sensitive:

```
- name: reporting-app
  type: custom
  hosts: [app01.example.com]
  user: appadm
  start:  "/opt/app/bin/appctl start"
  stop:   "/opt/app/bin/appctl stop"
  status: "/opt/app/bin/appctl status"
  up_pattern: "is running"
  down_pattern: "is not running|is stopped"
```

Write commands exactly as you would type them at a shell prompt on the tier host, and enclose them in double quotation marks. If a start command leaves a program running in the background, redirect its output, for example: **"nohup /opt/app/bin/server >/dev/null 2>&1 &"**. Otherwise, SAS_lsm waits up to 5 seconds for the command's output to end.

Enable Control of SAS® LASR™ Analytic Server

There are two methods for controlling LASR servers with SAS_lsm. The recommended method uses the LASROps utility, which controls LASR servers through the LASR API. The legacy SAS_lsm method is still available for MPP (multi-machine) LASR deployments, but it is more involved to configure.

Control LASR with the LASROps utility (recommended). Download and configure the LASROps utility from its GitHub project page (https://github.com/ajforeman/LASROps), following the instructions in its README. Test the utility on its own to verify that it can start, stop, and check the status of your LASR servers. Then add it to your configuration file as a custom tier:

```
- name: lasr
  type: custom
  hosts: [lasrhead01.example.com]
  start:  "/opt/sas/config/Utilities/LASROps/LASROps.sh start"
  stop:   "/opt/sas/config/Utilities/LASROps/LASROps.sh stop"
  status: "/opt/sas/config/Utilities/LASROps/LASROps.sh status"
  up_pattern: "is UP"
  down_pattern: "is DOWN"
  logdir: /opt/sas/config/Utilities/LASROps
```

Control LASR with the legacy SAS_lsm method (MPP LASR only). Generate the LASR control programs (mpp_start.sas, mpp_stop.sas, mpp_checklasr.sas, and the instance-specific configuration files) by following the "Enable Control of SAS LASR Analytic Server" section of the SAS_lsm 4.x guide, using the SAS_lsm_LASR.tar file from the SAS_lsm 4.x package. Then add one custom tier for each LASR port:

```
  - name: lasr-10011
    type: custom
    hosts: [sasapp-2.example.com]
    user: sasdemo
    start:  "/opt/sas94/SASHome/SASFoundation/9.4/sas
/usr/local/etc/SAS/LASR/mpp_start.sas -log /usr/local/etc/SAS/LASR/logs/10011 -sysparm
10011"
    stop:   "/opt/sas94/SASHome/SASFoundation/9.4/sas
/usr/local/etc/SAS/LASR/mpp_stop.sas -log /usr/local/etc/SAS/LASR/logs/10011 -sysparm
10011"
    status: "/opt/sas94/SASHome/SASFoundation/9.4/sas
/usr/local/etc/SAS/LASR/mpp_checklasr.sas -log /usr/local/etc/SAS/LASR/logs/10011
-sysparm 10011"
    up_pattern: "LASR UP"
    down_pattern: "LASR DOWN"
    logdir: /usr/local/etc/SAS/LASR/logs/10011
```

Validate the Configuration File

Run the validate command after every change to the configuration file:

```
sas_lsm validate -c /opt/sas/utilities/sas_lsm/prod.yaml
```

If the file is valid, SAS_lsm lists the tiers in order. If it is not valid, SAS_lsm lists every problem that it found, with line numbers, so that you can correct them all at once. For example:

```
configuration prod.yaml is not valid:
  - line 14: tier 2 (compute): unknown setting "paralel" (allowed: configdir, ...)
  - line 18: tier 3 (midtier): 'hosts' must list at least one host
```

Validation checks the file itself (spelling of settings, required values, and so on). It does not check the machines. That is the job of the doctor command, described below.

Follow these rules when you edit the file:

- Indent with spaces, never with tab characters.
- Write lists in square brackets, such as **[host1, host2]**, or as indented lines that begin with a dash.
- Enclose commands, and any value that contains a colon followed by a space or the # character, in double quotation marks. Inside double quotation marks, write \" for a quotation mark and \\ for a backslash.
- A # character starts a comment, unless it is inside quotation marks.

Check Every Host

The doctor command connects to every host in the configuration file and checks that SAS_lsm can work there:

```
sas_lsm doctor -c /opt/sas/utilities/sas_lsm/prod.yaml
```

It checks the following items:

- SAS_lsm is not running as root, and the report directory can be written.
- Each host can be reached without a password, and you are logged on as the expected account.
- The SAS scripts (sas.servers or MetadataServer.sh) exist and can be run in the configuration directory.
- For lsf-grid tiers, profile.lsf can be read and passwordless sudo works.
- UserExit scripts and directories exist and can be run.

Every failed check includes a suggested fix. Correct any failures, and run doctor again until all checks pass:

```
sasinst@sascomp01.example.com
  [ok  ] connect without a password
  [ok  ] logged in as the expected user - sasinst
  [FAIL] sas.servers is executable (2:compute)
         fix: check this tier's hosts and configdir (/opt/sas/config/Lev1)
```

Test a Dry Run

Before the first real run, use --dry-run to see exactly what SAS_lsm would do, in what order, without running anything:

```
sas_lsm start -c /opt/sas/utilities/sas_lsm/prod.yaml --dry-run
sas_lsm stop  -c /opt/sas/utilities/sas_lsm/prod.yaml --dry-run
```

Then check the current status of your deployment. This is a safe way to confirm that SAS_lsm reads the status of every tier correctly:

```
sas_lsm status -c /opt/sas/utilities/sas_lsm/prod.yaml
```

If a tier shows as UNKNOWN even though its machines can be reached, the status output did not contain the expected text. See "Troubleshooting."

USEREXIT FRAMEWORK

Capabilities

With the UserExit framework, you can run your own scripts at specific points while SAS_lsm starts, stops, or checks a tier. This gives administrators an easy way to perform additional tasks when SAS services are started and stopped, and a simple way to connect to third-party tools.

Using the UserExit framework is optional. You can run UserExit scripts at these points for each tier:

Event	When the Script Runs
pre_start	Before the tier is started
post_start_success	After the tier has started successfully
post_start_failure	After the tier has failed to start
pre_stop	Before the tier is stopped
post_stop_success	After the tier has stopped successfully
post_stop_failure	After the tier has failed to stop
pre_status	Before the tier's status is checked
post_status	After the tier's status is checked

A UserExit script can be marked as **required**. If a required script fails, SAS_lsm does not continue: a failed pre_ script stops the action before the tier is touched, and a failed post_ script marks the tier as failed, so later tiers are not started or stopped. If a script is not required, SAS_lsm reports a warning when it fails and continues.

Here are some examples of tasks that you can implement with UserExit scripts:

- Before a tier is started, verify that required network ports are not in use.
- After a tier is started successfully, start the SAS Deployment Agent service on that tier.
- After a tier fails to start, create a list of configuration files that were changed in the past day.
- Before a tier is stopped, e-mail users to tell them that SAS services are being restarted.
- After a tier is stopped successfully, back up the SASHOME and SAS configuration directories.
- After a tier fails to stop, list any orphaned processes on the host.

Configuration

List UserExit scripts in the tier's **userexits** section, under the event at which they should run. Scripts run in the order listed:

```
- name: compute
  hosts: [sascomp01.example.com, sascomp02.example.com]
  userexits:
    pre_stop:
      - /opt/sas/utilities/userexits/notify_users.sh
      - run: /opt/sas/utilities/userexits/drain_jobs.sh {host}
        required: true
    post_start_success:
      - run: /opt/sas/utilities/userexits/start_deployment_agent.sh
        on: all
```

A script can be listed by its path alone, or with the following options:

Option	Description	Default
run	The command to run, with any arguments. {host} is replaced with the host name.	**(required)**
required	true if a failure of this script must stop the SAS_lsm action.	**false**
on	first to run the script on the tier's first host only, or all to run it on every host of the tier.	**first**

UserExit scripts run on the tier's host, under the tier's account. The script must exist on that host and be executable by that account. The doctor command checks this.

Using a SAS_lsm 4.x UserExit Directory

If you already have UserExit scripts that follow the SAS_lsm 4.x naming convention, you can keep them where they are. Set **userexit_dir** in the tier to the directory on the tier's first host that contains the scripts:

```
- name: compute
  hosts: [sascomp01.example.com]
  userexit_dir: /opt/sas/userexits/compute
```

Script file names follow this syntax:

```
XX_[PRE|POST]_[START|STOP|STATUS]_[SUCCESS|FAILURE]_[REQUIRED]_identifying-name
```

- *XX* is a number that sets the order in which scripts run (01, 02, 03, and so on).

- PRE or POST, and START, STOP, or STATUS, are required.
- SUCCESS or FAILURE is optional, and applies to POST START and POST STOP scripts only. A POST START or POST STOP script **without** SUCCESS or FAILURE runs after either result.
- REQUIRED is optional, and marks the script as required.
- The *identifying-name* is any text that helps you recognize the script.

File names can be in uppercase, lowercase, or mixed case. Scripts in the directory run after any scripts listed in the tier's userexits section.

To see which scripts will run for each event, use the userexits command. It replaces the userExit_tester.bash script from SAS_lsm 4.x, and it checks the directory on the tier's host:

```
sas_lsm userexits -c /opt/sas/utilities/sas_lsm/prod.yaml --tier compute

tier 2:compute
  pre_stop             /opt/sas/userexits/compute/01_PRE_STOP_notify_users.sh   <-
/opt/sas/userexits/compute
  post_start_success   /opt/sas/utilities/userexits/start_deployment_agent.sh [all
hosts]   <- config
                       /opt/sas/userexits/compute/02_POST_START_agent.sh   <-
/opt/sas/userexits/compute
```

Custom Scripts

SAS_lsm determines whether a UserExit script succeeded from the script's **exit code**. An exit code of **0** means success. Any other exit code means failure.

> **Important:** This is a change from SAS_lsm 4.x, which also treated exit codes 1, 2, and 4 as success. If you have UserExit scripts that return these codes on purpose, change them to return 0, or use a wrapper script like the examples below.

If you want to run a third-party command that does not return exit code 0 on success, use a translation wrapper script:

Example 1: Decide success from the command's output

```
#!/bin/sh
OUTPUT=$(/path/to/some/service/serviceagent.sh start 2>&1)
case "$OUTPUT" in
  *"started successfully"*) exit 0 ;;
  *) echo "$OUTPUT"; exit 5 ;;
esac
```

Example 2: Translate exit codes

```
#!/bin/sh
/path/to/some/service/serviceagent.sh start
RC=$?
if [ $RC -eq 0 ] || [ $RC -eq 4 ]; then exit 0; fi   # 4 is acceptable for this tool
exit 5
```

Anything that a UserExit script prints is written to the SAS_lsm report, so you can include messages that will help you understand what happened.

RUNNING SAS_LSM COMMANDS

Every SAS_lsm action runs as one command. Only one action can run per command, and only one SAS_lsm action can run at a time for each configuration file.

Command	Action Performed	Statement of Action
`start --from TIER`	Start	Starts services, beginning at tier TIER and continuing to the last tier. If --from is omitted, starts all tiers.
`stop --to TIER`	Stop	Stops services, from the last tier down to and including tier TIER. If --to is omitted, stops all tiers.
`status`	Status	Checks the current status of services on all tiers.

TIER can be a tier number (1 is the first tier in the file) or a tier name, such as **--from midtier**.

SAS_lsm checks that the tiers that are not part of the action are in the proper state to support it. For example, suppose that you want to start a deployment from tier 3:

```
sas_lsm start --from 3 -c /opt/sas/utilities/sas_lsm/prod.yaml
```

To start the deployment at tier 3 safely, tiers 1 and 2 must already be running, and tiers 4 and later must be stopped. SAS_lsm checks the status of every other tier first. If any tier is not in the expected state, SAS_lsm reports a failure and starts nothing. If tier 3 itself is already running, SAS_lsm restarts it.

While it starts each tier, SAS_lsm waits until every host reports that its services are running before it starts the next tier. If a tier does not start, SAS_lsm does not start the tiers after it. Similarly, if a tier does not stop, SAS_lsm leaves the tiers before it running, so that, for example, the metadata server remains available while you investigate.

The table below shows some example commands and describes what they do:

Full Command	Statement of Action
`sas_lsm start -c prod.yaml`	Starts services on all tiers.
`sas_lsm start --from midtier -c prod.yaml`	Starts services beginning at the tier named midtier.
`sas_lsm stop -c prod.yaml`	Stops services on all tiers.
`sas_lsm stop --to 2 -c prod.yaml`	Stops services down to and including tier 2.
`sas_lsm status -c prod.yaml`	Checks the current status of services on all tiers.
`sas_lsm start -c prod.yaml --dry-run`	Shows what a start would do, without doing it.
`sas_lsm stop -c prod.yaml --collect-logs`	Stops all tiers, and collects logs if a tier fails to stop.

Command Options

Option	Applies To	Description
`-c FILE`	All commands	The configuration file. If omitted, SAS_lsm uses the SAS_LSM_CONFIG environment variable, and then sas_lsm.yaml in the current directory.
`--dry-run`	start, stop	Show what would be done, without doing it.
`--collect-logs`	start, stop, status	If a tier fails, collect its logs and create a support bundle.
`--no-userexits`	start, stop, status	Do not run any UserExit scripts for this run.
`-q`	start, stop, status	Do not write to the terminal. The report file is still written. Use this for scheduled runs.

The SAS_lsm 4.x command options still work, so existing scripts and cron entries continue to work after you switch to SAS_lsm 5 (with a converted configuration file):

SAS_lsm 4.x	SAS_lsm 5 Equivalent
`SAS_lsm -a 5 -c FILE`	`sas_lsm start --from 5 -c FILE`
`SAS_lsm -o 5 -c FILE`	`sas_lsm stop --to 5 -c FILE`
`SAS_lsm -s -c FILE`	`sas_lsm status -c FILE`
`-e`	`--collect-logs`
`-x`	`--no-userexits`

Exit Codes

Every SAS_lsm command returns an exit code that scripts and schedulers can test:

Exit Code	Meaning
`0`	Success.
`1`	An action failed, a status check after a start found tiers that are not fully running, or ssh-setup left a host that is not ready.
`2`	A start was refused because the dependency check failed. Nothing was changed.
`3`	The configuration file or the command is not valid.
`4`	The state of at least one tier could not be determined (for example, a host could not be reached).
`5`	Another SAS_lsm action is already running for this configuration file.

How SAS_lsm Decides Whether a Tier Is Running

SAS_lsm runs the tier's status command on each host and reads the output line by line:

Output Contains up_pattern	Output Contains down_pattern	Host State
Yes	No	UP
No	Yes	DOWN
Yes	Yes	PARTIAL (some services are up and some are down)
No	No	UNKNOWN (unexpected output, or the host could not be reached)

A tier is UP when all its hosts are UP, and DOWN when all its hosts are DOWN. A metadata-cluster tier follows the quorum rule described in "SAS_lsm Tier Types."

The status action reports the state of every tier. Its exit code depends on the last start or stop action. After a start, any tier that is not UP is a failure (exit code 1). After a stop, a stopped or partially stopped deployment is expected and is not a failure (exit code 0). If any tier is UNKNOWN, the exit code is 4.

CENTRALIZED REPORT OUTPUT AND LOG COLLECTION

SAS_lsm stores reports and collected logs on the SAS_lsm controller, in a directory for each configuration file: *status_root/config-name/*. For example, for the configuration file prod.yaml and the default status_root, the directory is /var/tmp/sas_lsm/prod/. The following table shows what you can find there:

Artifact	Description
`report.status`	Details of the most recent status action
`report.start`	Details of the most recent start action
`report.stop`	Details of the most recent stop action
`archive/`	Date-stamped copies of every start and stop report (when archive_reports is true)
`logfiles/`	Copies of the recent log files from the failed tier, organized by host and tier *
`logfiles.ANALYSIS`	The most frequent ERROR messages in the collected logs, and every ERROR line by file. Useful when you open a case with SAS Technical Support. *
`support-bundle-YYYYMMDD-HHMMSS.tar.gz`	A compressed archive of the reports, collected logs, and analysis, ready to send to SAS Technical Support *

* Created only when a tier fails and log collection is enabled (the --collect-logs option or the collect_logs setting).

The terminal shows a summary of each action. The report file contains the full details, including every command that SAS_lsm ran and the complete output of each command. When a tier does not behave as expected, the report file is the first place to look.

When a start or stop action fails, or a status action finds a problem after a start, and log collection is enabled, SAS_lsm does the following:

1. Finds the log files (files with the .log extension) under the failed tier's log directory that changed in the last *log_minutes* minutes (60 by default), up to 50 files on each host.
2. Copies the last *log_max_lines* lines (5,000 by default) of each file to the logfiles/ directory on the controller.
3. Analyzes the copied logs for ERROR messages, writes logfiles.ANALYSIS, and shows the 10 most frequent errors in the report.
4. Creates the support bundle.

If you cannot resolve the problem yourself, provide the support bundle to SAS Technical Support. Remove anything confidential first.

SCHEDULING SAS_LSM

SAS_lsm and cron

You can schedule SAS_lsm with cron. Run cron jobs as the SAS installer account, and use the -q option so that cron does not e-mail the terminal output:

```
# Check status every 10 minutes. E-mail is sent (via admin_email) only when something is
wrong.
*/10 * * * * /opt/sas/utilities/sas_lsm/sas_lsm status -q -c
/opt/sas/utilities/sas_lsm/prod.yaml

# Restart SAS every Sunday at 02:00.
0 2 * * 0 /opt/sas/utilities/sas_lsm/sas_lsm stop -q -c
/opt/sas/utilities/sas_lsm/prod.yaml && /opt/sas/utilities/sas_lsm/sas_lsm start -q
-c /opt/sas/utilities/sas_lsm/prod.yaml
```

SAS_lsm 5 prevents two actions from running at the same time for the same configuration file, whether they were started by cron or by a person. The second action ends immediately with exit code 5. You no longer need a separate copy of the configuration file for cron, as SAS_lsm 4.x recommended.

Because a status action after a deliberate stop is not a failure, a regular status check does not send false alarms during planned maintenance, as long as you stop SAS with SAS_lsm.

Starting SAS When the Machine Starts (Linux)

On Linux systems that use systemd, you can start and stop SAS with the SAS_lsm controller. Ask your system administrator to create the file /etc/systemd/system/sas-lsm.service on the controller:

```
[Unit]
Description=SAS 9.4 deployment (SAS_lsm)
After=network-online.target sshd.service
Wants=network-online.target

[Service]
Type=oneshot
RemainAfterExit=yes
User=sasinst
Environment=SAS_LSM_CONFIG=/opt/sas/utilities/sas_lsm/prod.yaml
ExecStart=/opt/sas/utilities/sas_lsm/sas_lsm start -q
ExecStop=/opt/sas/utilities/sas_lsm/sas_lsm stop -q
TimeoutStartSec=0
TimeoutStopSec=1800

[Install]
WantedBy=multi-user.target
```

Then enable it with the command **systemctl daemon-reload && systemctl enable sas-lsm**.

In a multi-machine deployment, all the other machines must be running and reachable by SSH before SAS_lsm starts. If they are not, the dependency check finds tiers in an UNKNOWN state, and SAS_lsm starts nothing. Make sure that the controller starts last, or schedule the start for some time after the machines start.

MIGRATING FROM SAS_LSM 4.X

Follow these steps to move from SAS_lsm 4.x to SAS_lsm 5:

1. Install SAS_lsm 5, as described in "Download and Install SAS_lsm." You can keep SAS_lsm 4.x installed while you test.
2. Convert your configuration file: **sas_lsm convert /path/to/SAS_lsm.cfg > prod.yaml**. Read the review notes that are displayed.
3. Run **sas_lsm validate**, then **sas_lsm ssh-setup -c prod.yaml** (hosts that already accept your existing key are skipped), then **sas_lsm doctor**, and then **sas_lsm start --dry-run**. Compare the plan with the way that SAS_lsm 4.x controls your deployment.
4. Run **sas_lsm status** and compare the result with **SAS_lsm -s**.
5. Change your cron entries and start-up scripts to use the new program and the new configuration file.

Your existing SSH keys continue to work. If the controller is now in FIPS mode and your SAS_lsm 4.x setup used an Ed25519 key, ssh-setup installs an ECDSA key alongside it.

What the convert Command Does

SAS_lsm 4.x	SAS_lsm 5
MAXTIERS and numbered TIER...[N] variables	An ordered list of tiers. Tiers are counted automatically.
MDCN tiers and the MDCV tier	One metadata-cluster tier
A tier whose start command is .../sas.servers start	A sas-servers tier with the matching configdir

SAS_lsm 4.x	SAS_lsm 5
Any other tier	A custom tier with the same commands
TIERSTOS / TIERSTAS	up_pattern / down_pattern
TIERNAME	A tier name that is easy to type (for example, Mid Tier becomes mid-tier)
INSTID, CONFIGDIR	defaults: user and configdir, with per-tier values where tiers differed
MAXRETRY, SLEEP	defaults: retries and retry_interval
STATUSROOT, SASADMIN, SSHOPTIONS	settings: status_root, admin_email, ssh_options
StrictHostKeyChecking=no	Replaced by StrictHostKeyChecking=accept-new
TIEREXIT	userexit_dir
TIERREST	Not needed. SAS_lsm stops and then starts the tier.

Changes in Behavior

Area	SAS_lsm 4.x	SAS_lsm 5
SSH direction	Described as required between all machines	Required from the controller to each machine only
SSH setup	Manual	ssh-setup command; one password per host; FIPS-aware
Per-tier values	A CONFIGDIR or INSTID value carried into the following tiers	Each tier uses its own value or the default
First tier of a start	Always restarted	Restarted only if it is not already stopped
Machines per tier	One	Any number, optionally in parallel
UserExit success	Exit codes 0, 1, 2, and 4	Exit code 0 only
UserExit file names	Lowercase only. POST scripts required SUCCESS or FAILURE.	Any case. POST scripts without SUCCESS or FAILURE run after either result.
Version check	Contacted GitHub on every run	Only when you run sas_lsm version --check
Wait between tiers	Fixed at 15 seconds	tier_delay, which can be set for each tier
Log collection	Copied whole log files and trimmed them	Copies the end of recent log files, and creates a support bundle
Exit codes	0 or 1	0 through 5

TROUBLESHOOTING

Symptom	Cause and Solution
ssh-setup lists a host as [needs key]	This is expected the first time. Continue, and enter that account's password when prompted.
ssh-setup lists a host as [problem]	The host name cannot be resolved, or the host or its SSH server cannot be reached. Correct the host name or network problem, and run ssh-setup again. Add --verbose to see the SSH program's own message.
ssh-setup lists a host as [host key]	The host's SSH key has changed since SAS_lsm first connected to it. Confirm with your system administrator that the change is expected, run ssh-keygen -R host on the controller, and run ssh-setup again.
ssh-setup summary shows a host as failed	The password was wrong, or the account cannot log on with a password. If the key was copied but logon still fails, check that ~/.ssh on that host has permission 700, ~/.ssh/authorized_keys has permission 600, and the home directory is not writable by the group.
Ed25519 keys are not allowed in FIPS mode	The controller is in FIPS mode and ~/.ssh/id_ed25519 exists. Run sas_lsm ssh-setup to install an ECDSA key. After that, the warning is harmless; move id_ed25519 to another directory to stop it.
ssh to sasinst@host failed (exit 255): Permission denied, during start, stop, or status	The SSH key is not installed for that account on that host. Run sas_lsm ssh-setup -c FILE.
A tier is UNKNOWN, but the host can be reached	The status output did not contain the up or down text. Look at the full output of the status command in report.status, and correct up_pattern or down_pattern.
sas.servers works when I log on, but not from SAS_lsm	SSH batch sessions do not read every profile file. Run ssh -o BatchMode=yes sasinst@host "/opt/sas/config/Lev1/sas.servers status" on the controller to reproduce the problem, and then set the missing variables in the account's ~/.bashrc.
The middle tier fails to start, but it is running a few minutes later	SAS_lsm stopped waiting too soon. Increase retries for the middle tier. If sas.servers start itself takes longer than 15 minutes, also increase command_timeout.
another SAS_lsm run (pid N) is active	Another action is running for the same configuration file (perhaps from cron). Wait for it to finish. If process N is not SAS_lsm, delete the .lock directory in the report directory.
An lsf-grid tier does not start or stop	Run sas_lsm doctor. The passwordless sudo check must pass. You might also need to disable requiretty in the sudoers file.
A command waits until command_timeout	A program started in the background kept the command open. Add >/dev/null 2>&1 & to background commands in custom tiers.

Symptom	Cause and Solution
ControlPath too long, or socket errors	Set ssh_multiplex: false in the settings section.
tab characters are not allowed	Replace tab characters in the configuration file with spaces.

If you need help, contact SAS Technical Support or open an issue on the SAS_lsm GitHub project page. Include the report file or the support bundle, with anything confidential removed.

APPENDIX: EXAMPLE CONFIGURATION FILES

Single Machine

Everything runs on one machine under the account that runs SAS_lsm, so no SSH setup is needed.

```
version: 1

defaults:
  user: sasinst
  configdir: /opt/sas/config/Lev1

tiers:
  - name: sas
    hosts: [localhost]
```

Metadata, Compute, and Middle Tier

```
version: 1

settings:
  status_root: /var/tmp/sas_lsm
  admin_email: [sasadmin@example.com]
  collect_logs: true

defaults:
  user: sasinst
  configdir: /opt/sas/config/Lev1

tiers:
  - name: metadata
    hosts: [sasmeta01.example.com]

  - name: compute
    hosts: [sascomp01.example.com, sascomp02.example.com]
    parallel: true
    userexits:
      pre_stop:
        - /opt/sas/utilities/userexits/notify_users.sh

  - name: midtier
    hosts: [sasmid01.example.com]
    configdir: /opt/sas/configmid/Lev1
    retries: 150
    command_timeout: 1800
```

Metadata Cluster, Platform LSF Grid, and Middle Tier

```
version: 1

settings:
  status_root: /var/tmp/sas_lsm
  collect_logs: true

defaults:
  user: sasinst
  configdir: /opt/sas/config/Lev1

tiers:
  - name: metadata
    type: metadata-cluster
    hosts: [meta1.example.com, meta2.example.com, meta3.example.com]

  - name: lsf
    type: lsf-grid
    parallel: true
    lsf:
      master: gridctl.example.com
      nodes: [node1.example.com, node2.example.com, node3.example.com]
      envdir: /opt/sas/lsf/conf
      serverdir: /opt/sas/lsf/10.1/linux2.6-glibc2.3-x86_64/etc
      pm_host: gridctl.example.com
      pm_envdir: /opt/sas/pm/conf
      pm_serverdir: /opt/sas/pm/10.2/linux2.6-glibc2.3-x86_64/etc

  - name: grid-compute
    hosts: [gridctl.example.com, node1.example.com, node2.example.com,
node3.example.com]
    parallel: true
    userexits:
      post_start_success:
        - run: /opt/sas/utilities/userexits/start_deployment_agent.sh
          on: all
      pre_stop:
        - run: /opt/sas/utilities/userexits/drain_grid_jobs.sh
          required: true

  - name: midtier
    hosts: [mid1.example.com]
    retries: 150
    command_timeout: 1800
```

APPENDIX: COMMAND REFERENCE

Command	Purpose
sas_lsm start [--from TIER] [--dry-run] [--collect-logs] [--no-userexits] [-q] -c FILE	Start tiers
sas_lsm stop [--to TIER] [--dry-run] [--collect-logs] [--no-userexits] [-q] -c FILE	Stop tiers
sas_lsm status [--collect-logs] [--no-userexits] [-q] -c FILE	Check all tiers
sas_lsm validate -c FILE	Check the configuration file

Command	Purpose
sas_lsm doctor -c FILE	Check SSH, accounts, paths, and permissions on every host
sas_lsm ssh-setup --hosts H1,USER@H2,... [--user USER] [--key-type TYPE] [--yes] [--verbose]	Create and install SSH keys on a list of hosts (no configuration file needed)
sas_lsm ssh-setup -c FILE [--key-type TYPE] [--yes] [--verbose]	Create and install SSH keys on every host in a configuration file
sas_lsm userexits [--tier TIER] -c FILE	List the UserExit scripts that will run
sas_lsm discover --hosts H1,H2,... --configdir DIR [--user USER]	Draft a configuration file from your machines
sas_lsm convert OLD.cfg	Convert a SAS_lsm 4.x configuration file
sas_lsm version [--check]	Show the version (--check also shows the newest release on GitHub)
sas_lsm help	Show command help